

第二章 WebGIS的网络基础

第1节 计算机网络的定义与组成

第2节 TCP/IP协议

第3节 Web基础知识

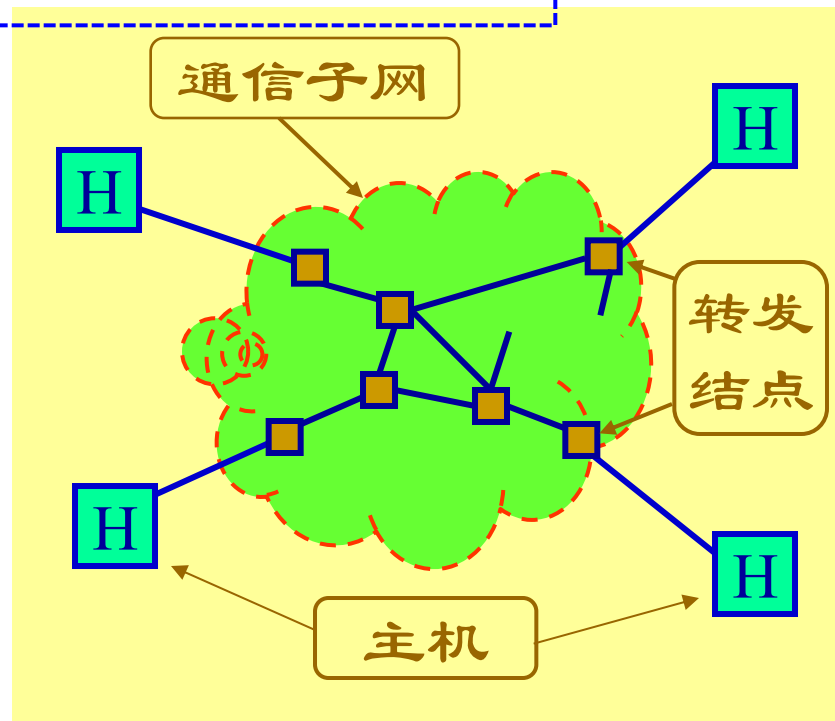
第2节 计算机网络的定义与组成

1. 计算机网络的概念

定义： 利用通信设备和线路将地理位置分散的、功能独立的自主计算机系统或由计算机控制的外部设备连接起来，在网络操作系统的控制下，按照约定的通信协议进行信息交换，实现资源共享的系统。

计算机网络的组成:

- ❑ 主机 (Host)
- ❑ 通信子网 (Communication Subnet)
 - 转发结点 (如路由器、交换机)
 - 通信链路
- ❑ 通信协议



第2节 TCP/IP协议

1.TCP/IP协议族

(1) 什么是协议？

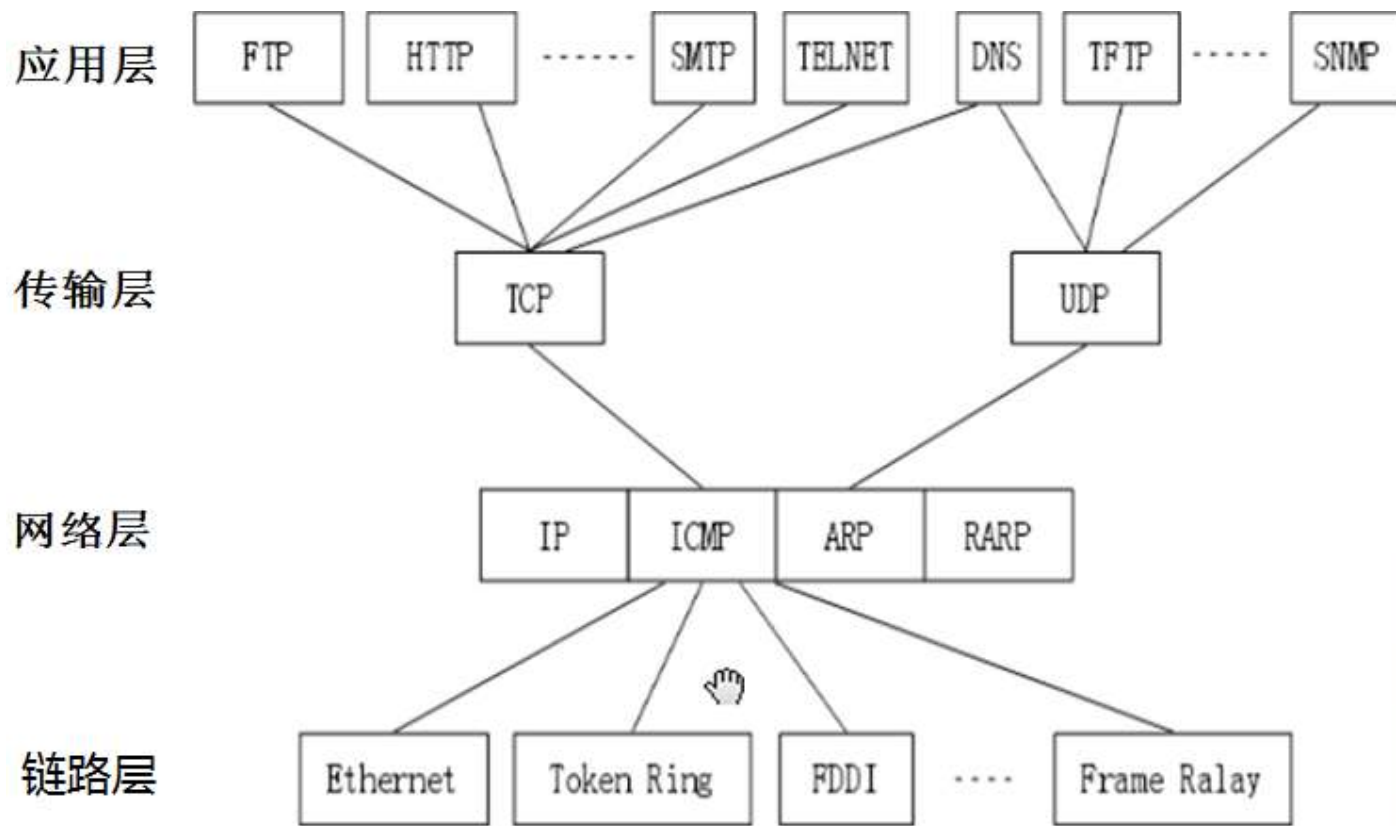
不同的硬件、操作系统之间的通信，所有的这一切都需要一种规则，而我们就把这种规则称为协议（Protocol）。

(2) TCP/IP

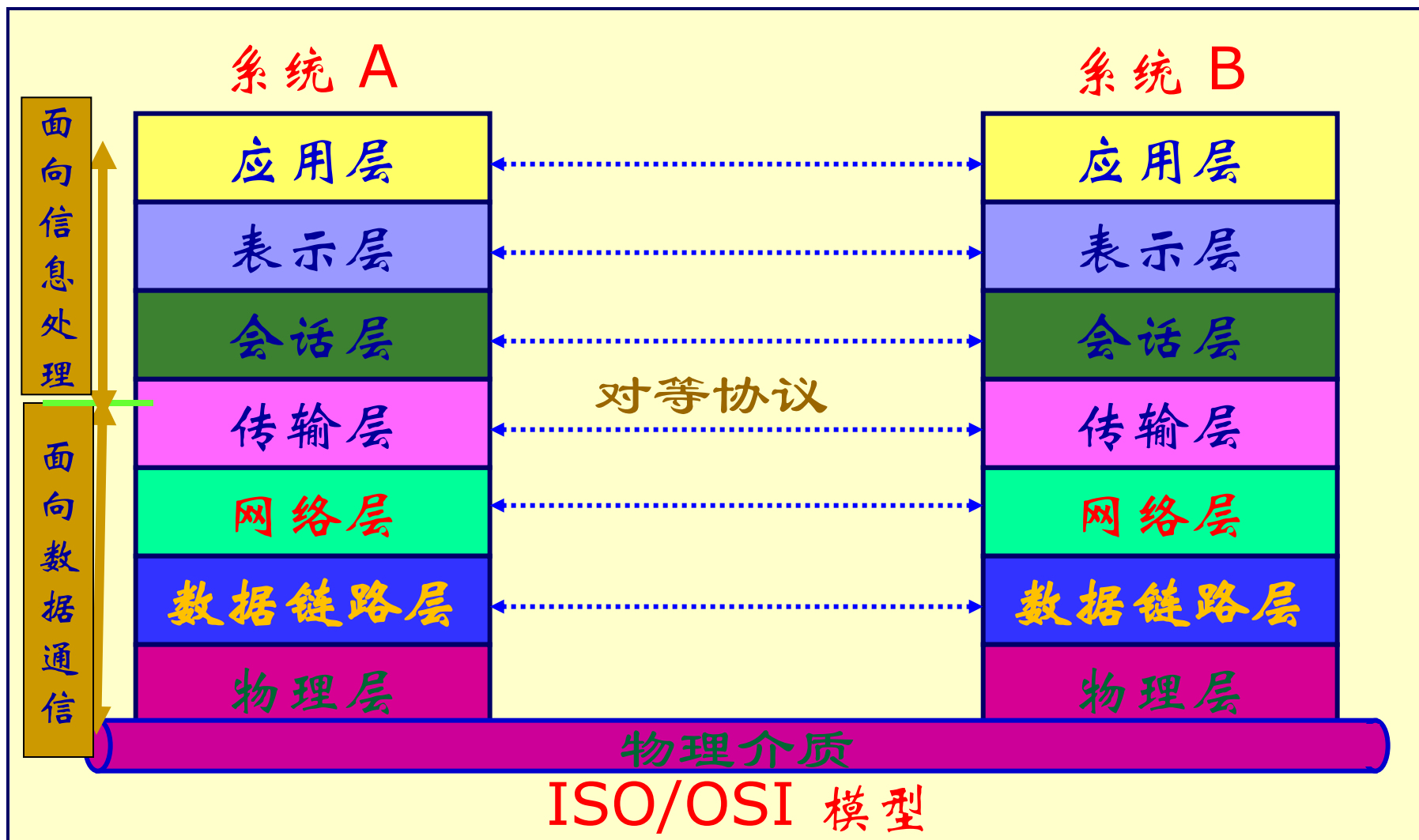
Transmission Control Protocol/Internet Protocol

狭义特指两个协议即传输控制协议TCP和网际协议IP；

广义指由多个与因特网相关的协议组成的TCP/IP参考模型。

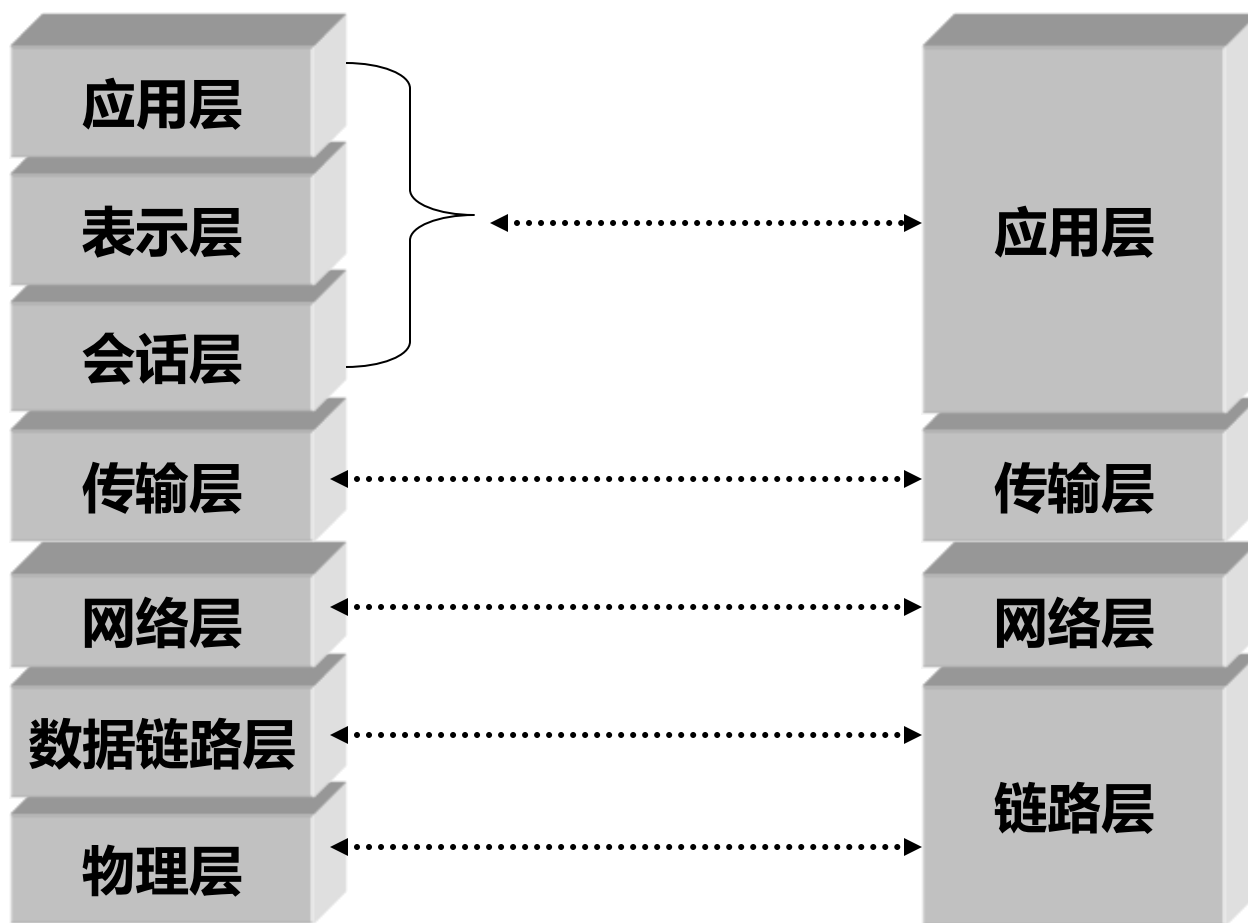


网络系统结构——开放系统互连参考模型



(3) TCP/IP与ISO/OSI的模型对比

区别：OSI对层的划分更精确，TCP/IP使用更宽的层定义；



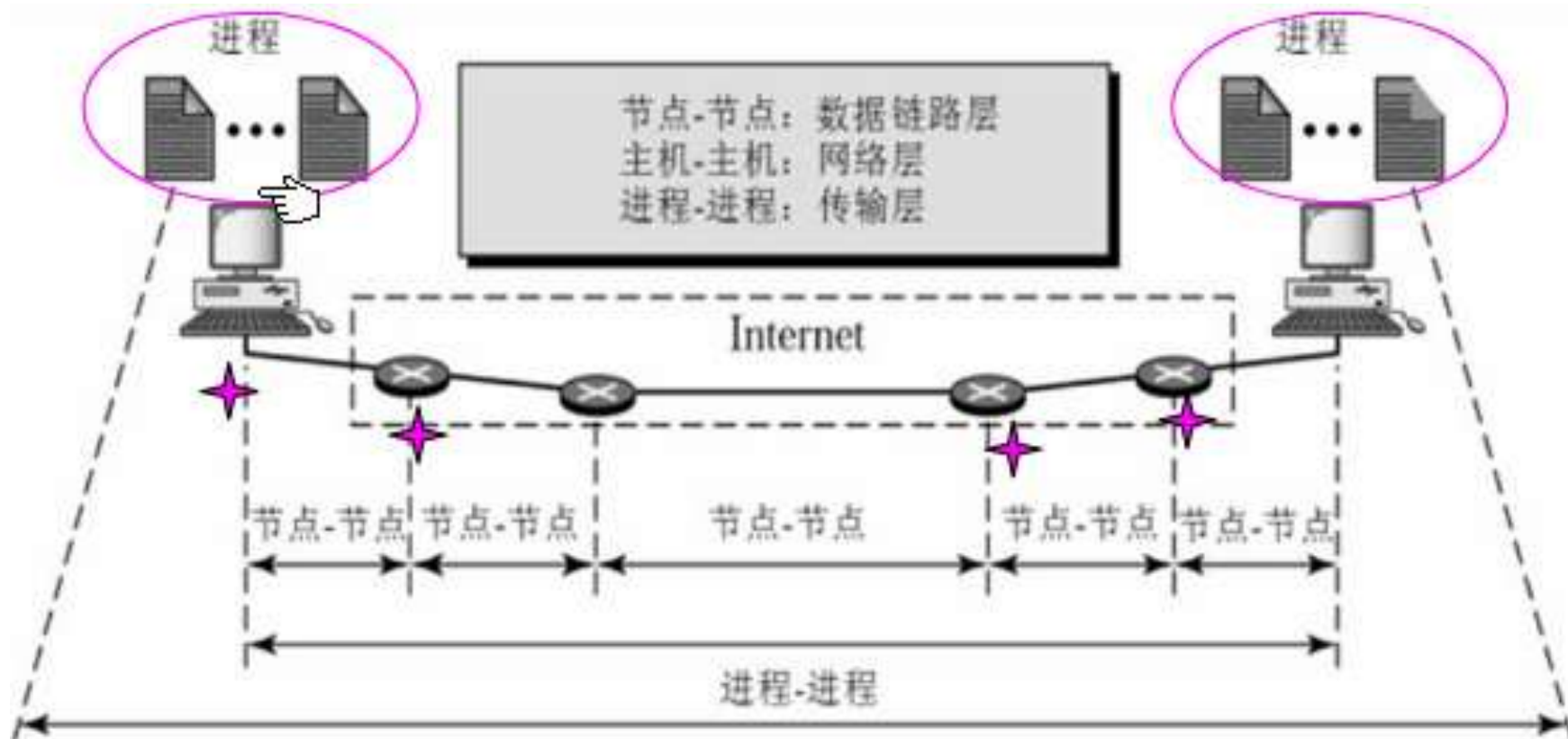
<i> 应用层

- 该层向用户提供一组常用的应用程序。不同的应用需要使用不同的应用层协议。
 - 1) 网络终端协议TELNET:用于实现远程登陆功能;
 - 2) 文件传输协议FTP:用于实现交互文件传输功能;
 - 3) 简单电子邮件传输协议SMTP: 用于电子邮件的传输;
 - 4) 域名服务DNS:用于实现域名到IP地址的转换;
 - 5) HTTP协议: 用于Web服务;
- 应用层协议负责将**网络传输的对象**转换成人们能识别的信息。

(2) 传输层

- 提供应用程序间的通信；
- 传输层协议主要是：传输控制协议TCP (Transmission Control Protocol) 和用户数据报协议UDP(User Datagram protocol) 。

指标	TCP	UDP
是否连接	面向连接	无连接
传输可靠性	可靠	不可靠
速度	较慢	较快
传输质量	较高	较差



传输层的主要用途和功能:

分段和**重组**: 将应用程序分割成若干段, 既可以保证所传输的大小符合传输介质的限制要求, 也可以确保不同应用程序发出的数据能在介质中多路传输。

(3) 网络层（又名网络互连层）

负责相邻**计算机**间的通信，其功能包括三方面：

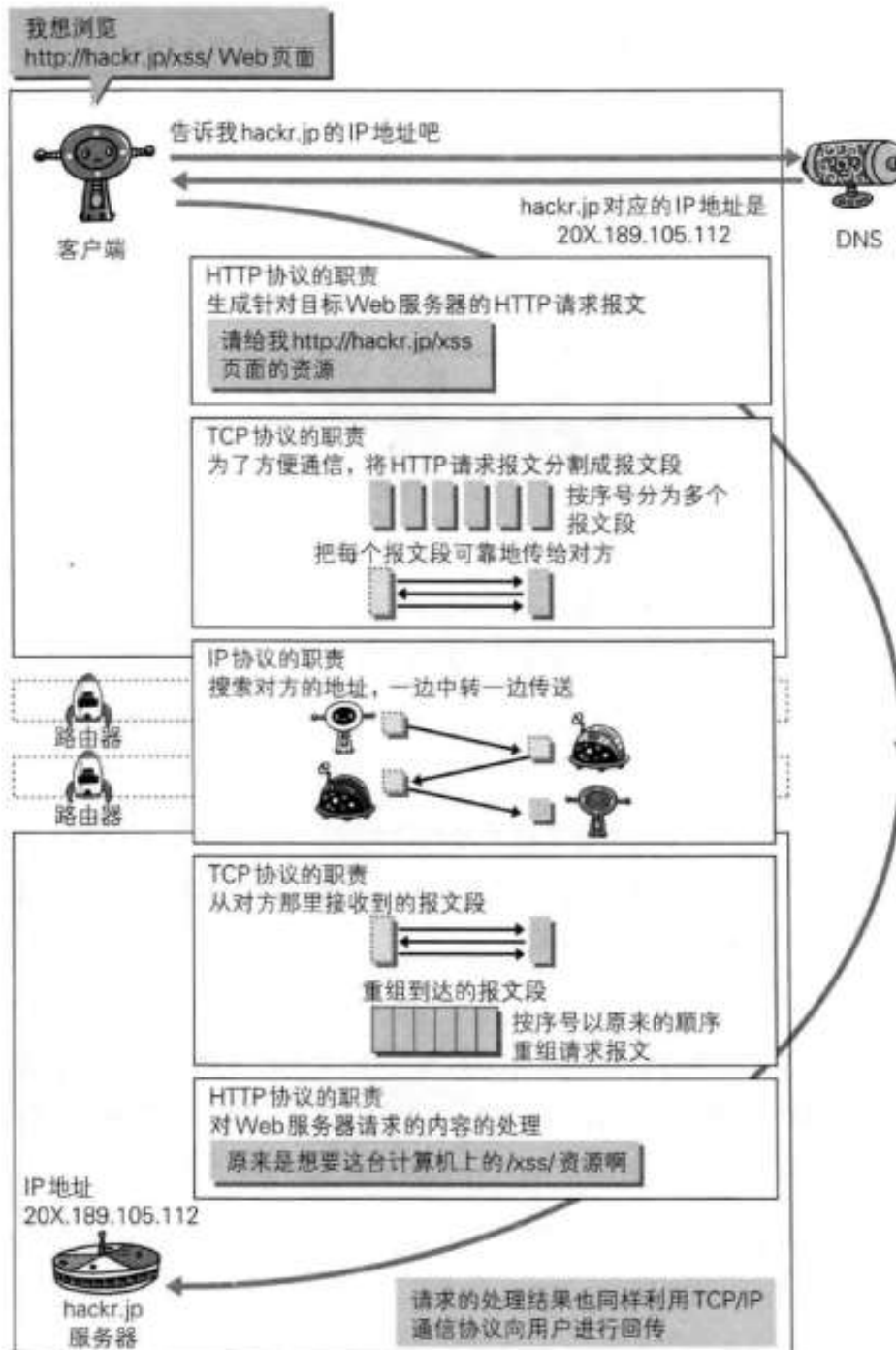
- 处理来自传输层的分组发送请求，收到请求后，将分组装入**IP**数据报，填充报头，选择去往信宿机的路径，然后将数据报发往适当的网络接口；
- 处理输入数据报：首先检查其合法性，然后进行寻径--假如该数据报已到达信宿机，则去掉报头，将剩下部分交给适当的传输协议；假如该数据报尚未到达信宿，则转发该数据报；
- 处理路径、流控、拥塞等问题；

(4) 链路层（又名网络接口层）

- 链路层是负责接收IP数据报，并把IP数据报组织成适合在具体物理网络中传输的帧，以及怎样在网络中传输帧；
- 从网络上接收物理帧，抽出IP数据报，交给IP层；

2. 与HTTP关系紧密的协议：IP、TCP和DNS

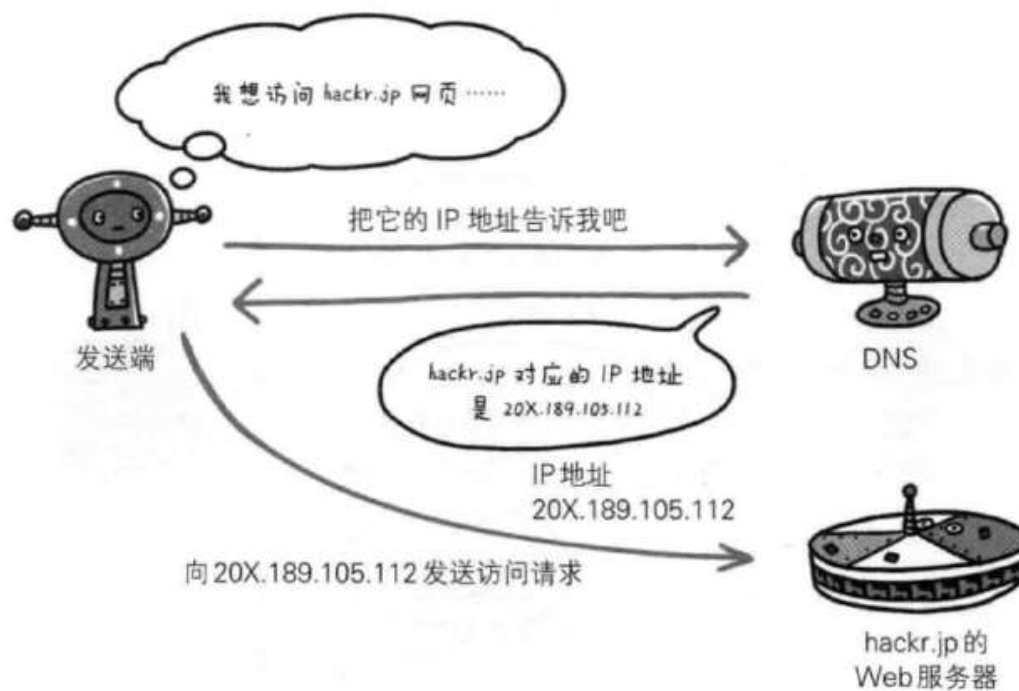
以HTTP协议举例，首先作为发送方的客户端，在**应用层（HTTP协议）**发出一个**想看某个Web页面的HTTP请求**。



■ DNS (Domain Name System)

计算机可以被赋予IP地址，也可以被赋予主机名或域名。比如：www.cumt.edu.cn;

DNS服务是和HTTP协议一样位于应用层的协议。它提供域名到IP地址之间的解析服务。



第3节 Web基础知识

1.HTTP协议

(Hypertext Transfer Protocol, 超文本传输协议)

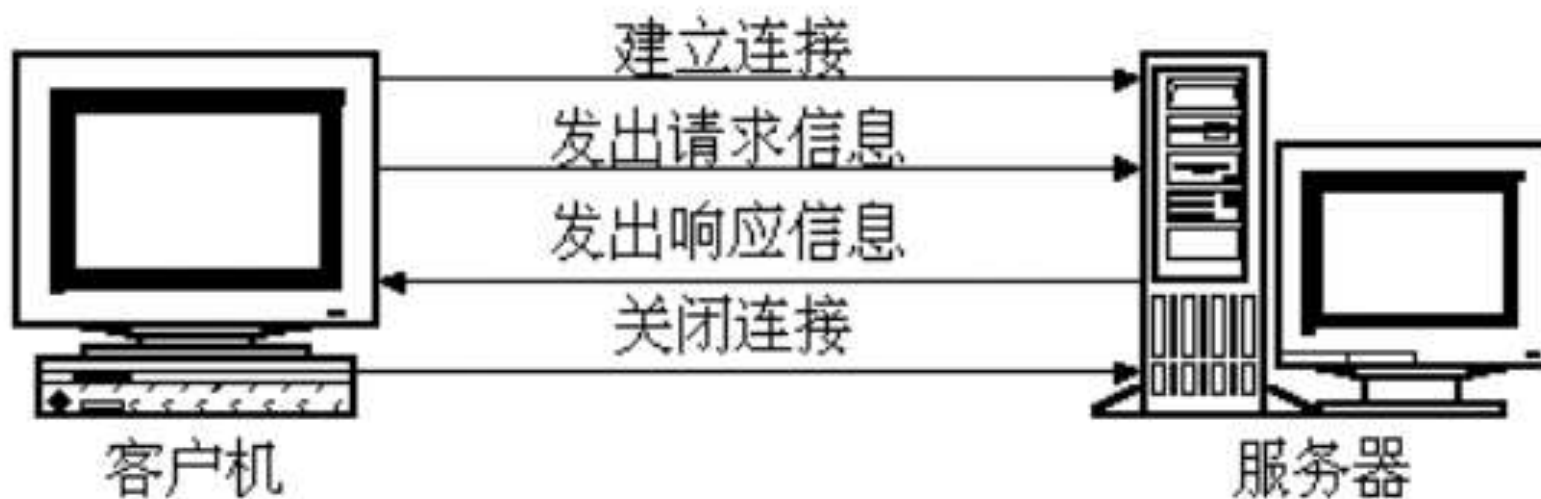
- 一种详细规定了Web服务器和浏览器之间互相通信的规则，在Internet上传送Web文档的数据传送协议；

HTTP协议的主要特点：

- 支持客户/服务器模式；
- 简单快速：由于HTTP协议简单，使得HTTP服务器的程序规模小，因而通信速度很快；
- 灵活：HTTP允许传输任意类型的数据对象；
- 无连接：HTTP限制每次连接只处理一个请求。服务器处理完客户机的请求，并收到客户的应答后，即断开连接；
- 无状态：HTTP协议是无状态协议。无状态是指协议对于事务处理没有记忆能力。缺少状态意味着如果后续处理需要前面的信息，则它必须重传，这样可能导致每次连接传送的数据量增大；

Http协议的请求响应过程:

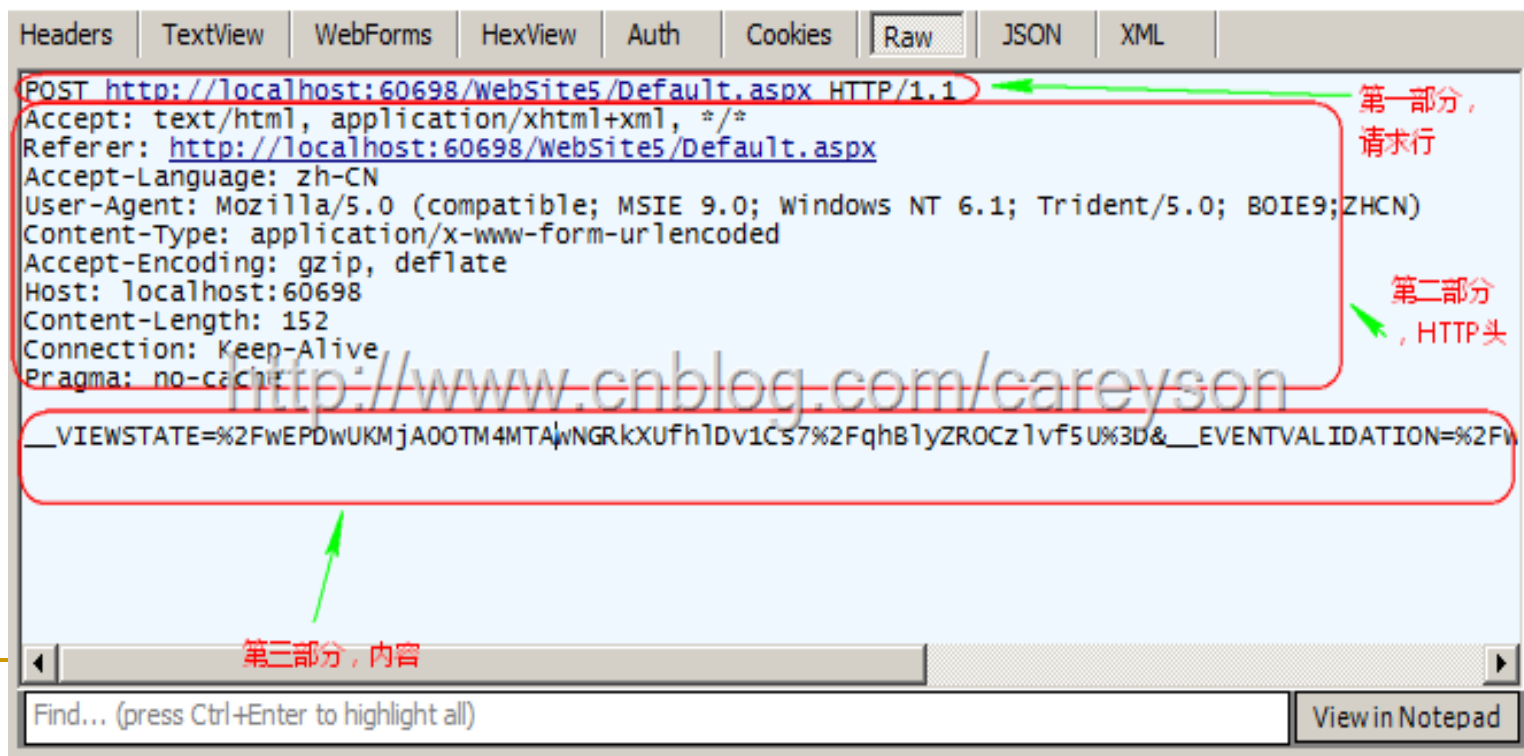
- 四个阶段：连接，请求，响应，断开；



■ HTTP请求

HTTP请求：Web客户端向Web服务器发送的信息，这个信息由如下三部分组成：

- 1.请求行：请求方法+请求网址+HTTP版本；
- 2.HTTP头
- 3.内容



■ HTTP响应

当Web服务器收到HTTP请求后，会根据请求的信息做某些处理，相应的返回一个HTTP响应。HTTP响应在结构上很类似于HTTP请求，也是由三部分组成，分别为：

1.状态行： HTTP版本+响应状态码+状态码的描述；

例子： HTTP/1.1 200 OK

状态码：

- 信息类 (100-199)
- 响应成功 (200-299)
- 重定向类 (300-399)
- 客户端错误类 (400-499)
- 服务端错误类 (500-599)

2.HTTP头

3.返回内容： HTTP请求所请求的信息。这个信息可以是一个HTML，也可以是一个图片。

MIME Type

■在浏览器中显示的内容有 HTML、有 XML、有 GIF、还有 Flash那么，浏览器是如何区分它们，决定什么内容用什么形式来显示呢？—— MIME Type，也就是[该资源的媒体类型](#)；

■在把输出结果传送到浏览器上的时候，浏览器必须启动适当的应用程序来处理这个输出文档。这可以通过多种类型 MIME（[多功能网际邮件扩充协议](#)）来完成。在HTTP中，MIME类型被定义在Content-Type header中。

- 每个MIME类型由两部分组成，前面是数据的大类别，例如声音audio、图像image等，后面定义具体的种类。
- 常见的MIME类型
 - 超文本标记语言文本 .html,.html text/html
 - 普通文本 .txt text/plain
 - RTF文本 .rtf application/rtf
 - GIF图形 .gif image/gif
 - JPEG图形 .jpeg,.jpg image/jpeg
 - au声音文件 .au audio/basic
 - MIDI音乐文件 mid,.midi audio/midi,audio/x-midi
 - RealAudio音乐文件 .ra, .ram audio/x-pn-realaudio
 - MPEG文件 .mpg,.mpeg video/mpeg
 - AVI文件 .avi video/x-msvideo
 - GZIP文件 .gz application/x-gzip
 - TAR文件 .tar application/x-tar

2.URL (Uniform Resource Locator)

要链接Web中不同计算机上的文档，需要定义一个统一的格式来描述互联网上的文件、文档片断或服务资源的地址，即URL。 URL是因特网上标准的资源地址。

例： <http://www.microsoft.com>， <ftp://ft.cdrom.com>等。

URL格式：

URL 的一般形式是：

<URL的访问方式>://<主机>[:<端口>]/<路径>

URI (Uniform Resource Identifier)

统一资源标识符

当用户从HTML网页链接到另外一个HTML网页时，依靠的是URI，[URI会把网页定位到世界上某台服务器上正确的位置](#)。URI是HTML网页的基本要素之一。

- 绝对URI地址：最前面是网络应用层的协议种类，后面跟 Web服务器地址和网页文件路径，最后可能会有具体文件的锚定位。

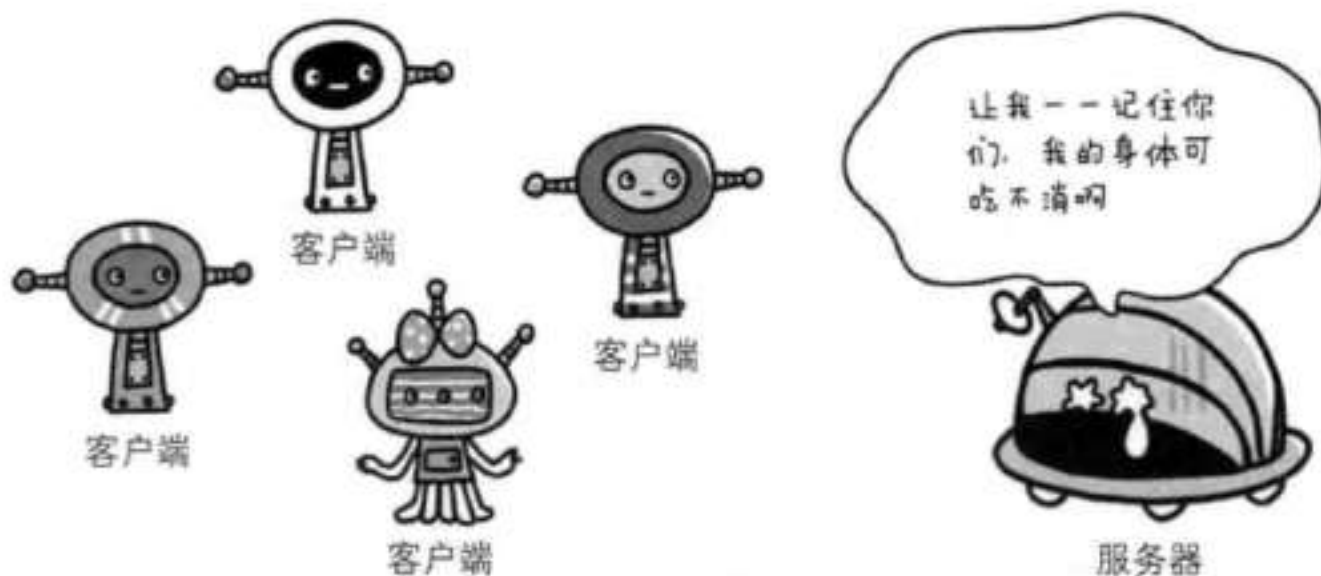
例如，<http://www.zeru.com/about/are.htm#a>

- 相对URI是根据当前文件位置，按照文件系统路径对其他网页文件进行定位。

../goods.html

3.使用Cookie的状态管理

- HTTP是**无状态协议**，它不对之前发生过的请求和响应的状态进行管理。也就是说，无法根据之前的状态进行本次的请求处理。
- 假设需要登录认证的Web页面，本身无法进行状态的管理（不记录已登录的状态），那么每次跳转新页面不是要再次登录，就是要在每次请求报文中附加参数来管理登录状态。



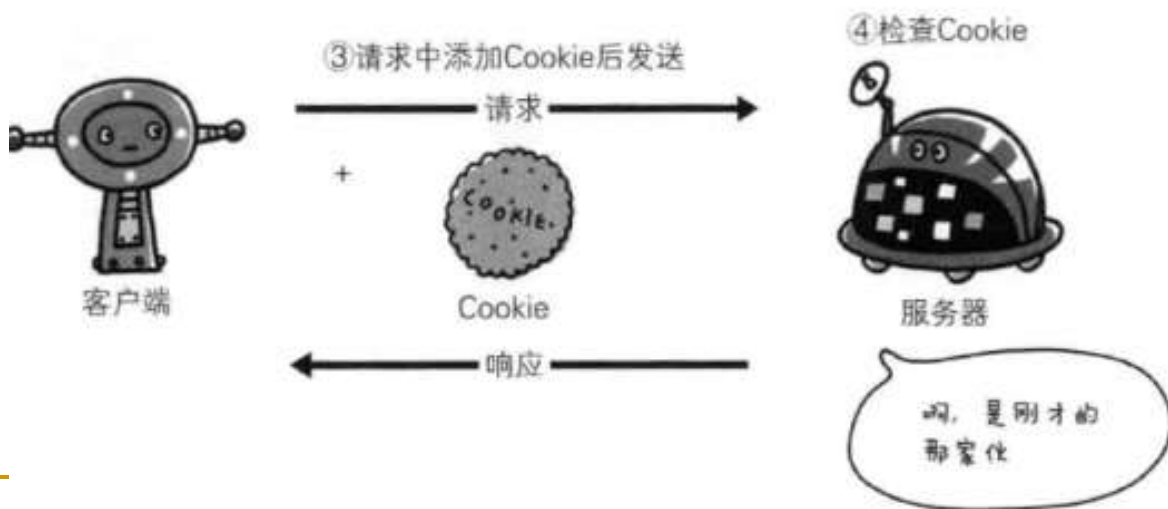
图：如果让服务器管理全部客户端状态则会成为负担

- 保留无状态协议这个特征的同时，又要解决类似的矛盾问题，于是引入了Cookie技术；
- Cookie技术通过在请求和响应报文中写入Cookie信息来控制客户端的状态；
- Cookie会根据从服务器端发送的响应报文内的一个叫做Set-Cookie的字段，通知客户端保存Cookie。当下次客户端再往该服务器发送请求时，客户端会自动在请求报文中加入Cookie值后发送出去。服务器端发现客户端发送过来的Cookie后，会检查究竟是从哪一个客户端发来的连接请求，然后对比服务器上的记录，最后得到之前的状态信息。

● 没有 Cookie 信息状态下的请求



● 第 2 次以后（存有 Cookie 信息状态）的请求



发生Cookie交互的情景

上图HTTP请求报文和响应报文的内容如下。

(1) 请求报文（没有Cookie信息的状态）

GET/reader/HTTP/1.1

Host: hackr.jp

***首部字段内没有Cookie的相关信息**

(2) 响应报文（服务器端生成Cookie信息）

HTTP/1.1 200 OK

Date: The, 12 Jul 2012 07:12:20 GMT

Server:Apache

<Set-Cookie:sid=1342077140226765;path=/;expires=Web,→10-Oct-12 07:12:20 GMT>

Content-Type: text/plain;charset=UTF-8

(3) 请求报文（自动发送保存的Cookie信息）

GET/image/HTTP/1.1

Host:hackr.jp

Cookie:sid=1342077140226765

图解HTTP

OWASP 日本分会主席 【日】上野 宣 著
于均良 译



HTTP扩展学习链接:

<https://www.cnblogs.com/road2master/p/16865658.html>